Introducción al C++

Notas respecto a la forma de realizar los ejercicios:

- La definición de la clase se debe realizar en un .h, mientras que la implementación de la misma se debe realizar en el punto cpp (o extensión similar), excepto se solicite lo contrario.
- Los ejercicios, son evolutivos y en cada serie, un ejercicio depende del ejercicio anterior.
- El código debe ser siempre el menor posible. Esto significa que si una porción de código se repite, realizar una macro o encapsularlo en una función. De igual manera si el constructor o destructor por omisión son suficientes, no implementarlos. En todos los casos analizar si la codificación realizada es realmente necesaria.
- Siempre que sea posible, aunque el enunciado no lo exprese, utilizar el atributo **const**.
- Utilizando el número de cada ejercicio elegir la rama del repositorio indicada para poder evaluar lo realizado. Al correr el programa se puede observar la salida obtenida y la de la solución del ejercicio.

Ejercicio 1

Definir e implementar una clase Punto, que posea los elementos públicos y privados:
Variables miembro privadas:

mx y **my** - del tipo **double**, que representan las coordenadas (x; y) del punto.

Métodos miembro públicos:

setPunto: recibe los valores de x e y en dos variables **double**.
getPunto: devuelve el valor del punto (en formato class Punto). En formato **inline**
setX y **setY**; que dan valor a x e y. En formato **inline**
getX y **getY**; que devuelven los valores de x e y. En formato **inline**

La creación del objeto debe permitir o no, las siguientes expresiones:

Punto pa; // **mx** y **my** se inicializan en cero.
Punto pb (23.3,56.8); // **mx** se inicializa con 23.3 y **my** con 56.8.

La inicialización del objeto con un solo parámetro debe dar error de sintaxis, por ejemplo la línea:

Punto pc (34.4); // error de sintaxis

Ejercicio 2

Modificar la clase del punto anterior, de manera de verificar que los valores de **x** e **y** se encuentren siempre dentro del rango -1000 a 1000, caso contrario su valor debe limitarse a estos valores. *(Utilice un método teniendo en cuenta el concepto de **Encapsulamiento**)*

Ejercicio 3

Modificar el punto anterior de manera que acepte también el Método "**setPunto** (**pa**)" en donde **pa** es un objeto **Punto** y acepte la creación del objeto de manera que la expresión "**Punto pc** (**34.4**)" sea aceptada asignando el valor pasado como parámetro a la coordenada **x** y asigne el valor cero a la coordenada **y**.

Ejercicio 4

Modificar el punto anterior de manera que acepte la creación de un objeto de la siguiente manera:
Punto pd (**pc**); // donde **pc** es un objeto **Punto**.
Punto pe = **pc**;

Ejercicio 5

Modificar el punto anterior para permitir la suma (+), resta (-) y asignación (=) de objetos tipo **Punto** de manera tal que dichas sobrecargas tengan un solo argumento.
Nota: tener en cuenta que las operaciones de deben controlar que los valores de **x** e **y**, no desborden el rango de +/- 1000.

Preguntas:
-¿Es necesario sobrecargar el operador de asignación (=) para obtener el resultado buscado? ¿Por qué?
- Justifique la ejecución de la línea `q=r+47;` (cómo se produce la suma entre el objeto **Punto** y el objeto **int**, suponiendo que mantuvo la consigna de escribir lo menos posible y no sobrecargó la suma de **Punto** con **int**).

Ejercicio 6

Modificar el punto anterior para permitir las siguientes operaciones:

pa = 4 + **pb**; //Donde **pa** y **pb** son objetos del tipo **Punto**
pb = 3 - **pa**;

Ejercicio 7

Modificar el punto anterior para permitir la comparación entre objetos tipo Punto: **igualdad** (**==**), **desigualdad** (**!=**), **mayor** (**>**) y **menor** (**<**), sobrecargar el flujo de salida y entrada (**<<** y **>>**).
En el caso de la sobrecarga de **mayor** (**>**) y **menor** (<), se debe comparar la distancia que existe al centro de coordenadas (0;0). *Para ello es recomendable incluir la biblioteca **cmath** para el uso de **sqrt** ( ).*

Ejercicio 8

Agregar a la clase **Punto**, del ejercicio anterior, dos métodos públicos del tipo inline que indiquen la cantidad de instancias del tipo objeto que han sido creadas *(**getCantCreada( )**)* y cuántas se encuentran en memoria *(**getCantExistente( )**)*.
Nota: los contadores deben ser miembros privados de la clase.

Ejercicio 9

Realice una herencia de la clase **Punto** para realizar una clase **Circulo** donde las coordenadas **x** e **y** sean su centro y posea un atributo **double** que sea su **radio.** Además sobrecargue el operador de asignación.

Dicha clase debe permitir la creación de objetos de la siguiente manera:

Circulo ca; // **x** = 0, **y** = 0 **radio** = 1
Circulo cb(**3**); // **x** = 0, **y** = 0 **radio** = 3
Circulo cc(**4**,**2**); // **x** = 2, **y** = 0 **radio** = 4
Circulo cd(**5**,**2**,**3**); // **x** = 2 **y** = 3 **radio** = 5

.
Nota: la sobrecarga del operador de extracción de flujo está desarrollada en el archivo "main.cpp" pero necesita su declaración para funcionar correctamente.

Ejercicio 10

A través de una composición, desarrolle la clase **vecPunto** que permita crear arreglos de objetos tipo **Punto**.
Dicha clase deberá aceptar las siguientes sentencias:

```
vecPunto va;
vecPunto vb(LARGO_VECTOR);          //Por ejemplo 4 para un arreglo de 4 puntos
vecPunto vc = vb;
cout << va[3];                      //Lo cuál permitiría imprimir el tercer Punto del vector va
```

Ejercicios Adicionales:

Esta sección ocupa el lugar de práctica adicional sobre temas que ya fueron cubiertos en la guía y sirven como refuerzo sobre aquellos temas que no se hayan podido fijar.

Adicional 1 - Métodos y atributos estáticos

En la clase Punto:
Agregar un método público "**setLimites** (**double**,**double**)", que modifique el rango válido de **x** e **y** de la clase.
Esto significa que el rango de todos los objetos existentes y de los objetos por crear se verán afectados por este método.
Este método no debe modificar los valores de coordenadas **x** e **y** establecidas previamente. (No importa que éstos queden fuera de rango).

El primer parámetro corresponde al límite inferior y el segundo al superior. Si el límite inferior no es menor al superior, se debe omitir el cambio del rango.

Implementar además las funciones "**getLimiteSup**" y "**getLimiteInf**", del tipo inline, para saber cuáles son los valores de estos límites. Por omisión, los límites son +/- 1000.

Pregunta: ¿por qué no se puede definir las funciones **getLimiteSup** y **getLimiteInf** como **const**?

Adicional 2 - Sobrecarga de Operadores

En la clase Punto:
Sobrecargar el pre y post incremento de manera que incremente en una unidad tanto el valor de **x** como de **y**.

Adicional 3 - Sobrecarga de Operadores y Herencia

En la clase Circulo:
Sobrecargue los operadores del ejercicio 7, para el caso de **mayor** y **menor** compare la superficie de los círculos.

Adicional 4 - Sobrecarga de Operadores

Desarrolle los métodos de la clase String propuesta en “string.h” que puede ser encontrada en la rama de este ejercicio.